NECTY

V1 FULL-STACK DEVELOPER BUILD DOCUMENT

Production-Ready Engineering Blueprint

Next.js · Supabase · Airtable · Make.com · Claude API · Stripe · Vercel

Confidential — For Developer Use Only

Port + Pilot Inc. | NECTY Platform | Version 1.0

SECTION 0 PRODUCT OVERVIEW & DEVELOPER ORIENTATION

What You Are Building

NECTY V1 is an AI-powered opportunity intelligence platform for local service businesses. It scrapes public conversations across platforms, detects high-intent buying signals, classifies and scores them as opportunities, generates AI-powered comment and DM suggestions, allows users to copy and paste responses manually, tracks lightweight performance feedback, and surfaces messaging intelligence insights.

This is not a CRM. It is not an automation platform. It is not a social scheduler. It is not a reputation management suite. It is not an enterprise analytics dashboard. It is a fast, focused, opportunity-first product that turns public conversations into booked clients.

> **Core V1 Workflow**
>
> 1. Scrape public conversations (Apify via Make.com)
> 2. Detect high-intent buying signals (Claude API Agent 1)
> 3. Classify and score opportunities (Airtable + Make.com)
> 4. Generate AI-powered comments and DMs (Claude API Agent 2)
> 5. User copies and pastes responses manually (Next.js UI)
> 6. Track lightweight performance feedback (Supabase + Make.com webhook)
> 7. Surface messaging intelligence insights (Insights page)

Who Owns What

Layer	Technology	Who Builds It	Responsibility
Frontend	Next.js 14 + Tailwind CSS	Full-stack developer	All UI pages, components, routing, state, API calls
Auth	Supabase Auth	Full-stack developer	Sign up, login, session, JWT, middleware
Primary DB (V1)	Supabase (Postgres)	Full-stack developer	Users, workspaces, messages, performance data, settings
Intelligence DB	Airtable	Automation contractor	Demand signals, opportunities, AI outputs, trends
Automation	Make.com	Automation contractor	Scraping, classification, generation, routing
Scraping	Apify	Automation contractor	Multi-platform conversation scraping
AI Engine	Claude API	Automation contractor	Signal classification, message generation, insights
Payments	Stripe	Full-stack developer	Subscription plans, webhooks, plan gating

Layer	Technology	Who Builds It	Responsibility
Hosting	Vercel	Full-stack developer	Deployment, env vars, domain, edge functions

The Airtable / Supabase Split (Critical to Understand)

V1 uses both Airtable and Supabase. This is intentional. Airtable is where the automation contractor stores scraped signals and AI-generated opportunities — it is the intelligence pipeline's database. Supabase is where the frontend stores everything the user controls: their account, settings, message tracking actions, and performance data.

Data Type	Lives In	Why
Scraped demand signals	Airtable	Written by Make.com automation — no frontend writes to this
AI-generated opportunities	Airtable	Created by Claude API pipeline in Make.com
AI-generated comment/DM text	Airtable	Generated and stored by Agent 2 in Make.com
User accounts + auth	Supabase	Supabase Auth manages all identity
Workspace settings	Supabase	User-editable, frontend writes directly
Message copy/send tracking	Supabase	Frontend triggers, stored in Supabase messages table
Performance feedback	Supabase	User marks replied/clicked — frontend writes to Supabase
Insights surface data	Supabase (synced from Airtable)	Make.com syncs weekly insight summaries to Supabase
Stripe subscription data	Supabase	Stripe webhooks write plan + status to Supabase

Developer-Only Tasks — Do Not Delegate

Supabase setup and all Row-Level Security (RLS) policies
Stripe integration, webhook endpoint, plan gating middleware
Vercel project setup, environment variable configuration, domain
Supabase Auth — sign up, login, password reset, session management
All Next.js API routes and server actions
Airtable read API integration from the frontend (read-only)
Make.com webhook endpoint configuration on the frontend side

SECTION 1 TECH STACK & PROJECT STRUCTURE

Confirmed Tech Stack

Category	Technology	Version	Purpose
Frontend framework	Next.js	14 (App Router)	React SSR/SSG + API routes + server actions
UI library	React	18	Component model
Styling	Tailwind CSS	3.4+	Utility-first styling
Component library	shadcn/ui	Latest	Accessible, unstyled base components
Icons	Lucide React	Latest	Icon set throughout UI
Database (primary)	Supabase (Postgres)	Latest	Auth + user data + settings + tracking
Database (intelligence)	Airtable	Latest	Opportunity pipeline — read from frontend
Auth	Supabase Auth	Built into Supabase	JWT-based session management
Payments	Stripe	Latest SDK	Subscription billing
AI	Anthropic Claude API	claude-sonnet-4-6	Comment/DM generation (via Make.com)
Automation	Make.com	N/A	Scrape → classify → generate pipeline
Scraping	Apify	N/A	Multi-platform conversation scrapers
Deployment	Vercel	Latest	Hosting + edge functions + env vars
Email	Resend	Latest	Transactional email (auth, notifications)
State management	Zustand	Latest	Lightweight global state for UI
Data fetching	TanStack Query	v5	Server state, caching, refetching
Forms	React Hook Form + Zod	Latest	Form state + validation
Date handling	date-fns	Latest	Date formatting throughout UI

Recommended Folder Structure

```
necty/
├── app/                          # Next.js App Router
│   ├── (auth)/                   # Auth route group (no sidebar)
│   │   ├── login/page.tsx
│   │   ├── signup/page.tsx
│   │   └── onboarding/page.tsx
│   ├── (dashboard)/              # Authenticated route group
```

```
│   │   ├── layout.tsx            # Sidebar + top nav wrapper
│   │   ├── dashboard/page.tsx
│   │   ├── opportunities/page.tsx
│   │   ├── comments/page.tsx
│   │   ├── dms/page.tsx
│   │   ├── insights/page.tsx
│   │   ├── sources/page.tsx
│   │   ├── settings/page.tsx
│   │   └── help/page.tsx
│   ├── api/                      # API Routes
│   │   ├── webhooks/             # Stripe + Make.com webhooks
│   │   │   ├── stripe/route.ts
│   │   │   └── make/route.ts
│   │   ├── airtable/             # Airtable proxy reads
│   │   │   ├── opportunities/route.ts
│   │   │   ├── comments/route.ts
│   │   │   ├── dms/route.ts
│   │   │   └── insights/route.ts
│   │   ├── tracking/             # Copy/send tracking
│   │   │   └── route.ts
│   │   └── stripe/               # Stripe session + portal
│   │       └── route.ts
│   ├── globals.css
│   └── layout.tsx                # Root layout
├── components/
│   ├── ui/                       # shadcn/ui base components
│   ├── layout/                   # Sidebar, TopNav, MobileNav
│   ├── dashboard/                # Dashboard-specific components
│   ├── opportunities/            # OpportunityCard, OpportunityList, etc.
│   ├── comments/                 # CommentCard, CommentAngles, etc.
│   ├── dms/                      # DMCard, DMAngles, etc.
│   ├── insights/                 # InsightCard, PerformanceTable, etc.
│   ├── sources/                  # SourceCard, SearchConfig, etc.
│   ├── settings/                 # BusinessProfile, AIPreferences, etc.
│   ├── shared/                   # KPICard, FilterBar, EmptyState, etc.
│   └── onboarding/               # OnboardingStep, ProgressBar, etc.
├── lib/
│   ├── supabase/                 # Supabase client + server + middleware
│   │   ├── client.ts             # Browser client
│   │   ├── server.ts             # Server component client
│   │   └── middleware.ts         # Session refresh
│   ├── airtable/                 # Airtable API client (server-side only)
│   │   ├── client.ts
│   │   └── queries.ts
│   ├── stripe/                   # Stripe client + helpers
│   │   ├── client.ts
│   │   └── plans.ts
│   └── utils/                    # cn(), formatDate(), etc.
├── hooks/                        # Custom React hooks
│   ├── useOpportunities.ts
│   ├── useComments.ts
│   ├── useDMs.ts
│   ├── useInsights.ts
│   ├── useWorkspace.ts
│   └── useTracking.ts
├── store/                        # Zustand stores
│   ├── workspace.ts
│   ├── filters.ts
│   └── notifications.ts
```

```
├── types/                      # TypeScript types
│   ├── opportunity.ts
│   ├── message.ts
│   ├── workspace.ts
│   └── supabase.ts             # Generated from Supabase CLI
├── middleware.ts                # Auth middleware (Supabase session)
├── .env.local                   # Local environment variables
└── next.config.js
```

Naming Conventions

- Files: kebab-case for files (opportunity-card.tsx), PascalCase for components (OpportunityCard)
- Supabase tables: snake_case (workspaces, demand_signals, message_tracking)
- TypeScript types: PascalCase (Opportunity, DMAngle, WorkspaceSettings)
- API routes: REST-style nouns (/api/airtable/opportunities, /api/tracking)
- Zustand stores: camelCase files, PascalCase store names (useWorkspaceStore)
- Hooks: useNoun pattern (useOpportunities, useFilters, useTracking)
- Environment variables: NEXT_PUBLIC_ prefix for client-safe, no prefix for server-only

SECTION 2 SUPABASE DATABASE SCHEMA

Supabase stores everything the user controls. Airtable stores everything the automation pipeline produces. The frontend reads from both but only writes to Supabase. The automation contractor writes to Airtable. Make.com syncs select data from Airtable to Supabase (insights, opportunity counts for the dashboard).

Table: workspaces

One record per business account. Created during onboarding. All other Supabase tables link here via workspace_id.

```
CREATE TABLE workspaces (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_id        UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  business_name   TEXT NOT NULL,
  industry        TEXT NOT NULL,
  custom_industry TEXT,
  city            TEXT NOT NULL,
  state           TEXT NOT NULL,
  service_area_mi INTEGER DEFAULT 25,
  booking_link    TEXT,
  website_url     TEXT,
  phone           TEXT,
  description     TEXT,
  specialties     TEXT[],                  -- array of specialty tags
  logo_url        TEXT,
  years_in_biz    INTEGER,
  airtable_client_id TEXT,                 -- links to Airtable control_panel.client_id
  stripe_customer_id TEXT,
  plan            TEXT DEFAULT 'trial',    -- trial | pro | agency
  plan_status     TEXT DEFAULT 'active',   -- active | past_due | cancelled
  created_at      TIMESTAMPTZ DEFAULT now(),
  updated_at      TIMESTAMPTZ DEFAULT now()
);
```

Table: workspace_members

Supports team access. The owner always has full_access. Invited members get editor or viewer roles.

```
CREATE TABLE workspace_members (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id  UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
  user_id       UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  role          TEXT NOT NULL DEFAULT 'viewer',  -- owner | editor | viewer
  invited_email TEXT,
  accepted      BOOLEAN DEFAULT false,
  invited_at    TIMESTAMPTZ DEFAULT now(),
  UNIQUE(workspace_id, user_id)
);
```

Table: ai_preferences

Stores each workspace's AI messaging configuration. One record per workspace. Created with defaults on onboarding.

```
CREATE TABLE ai_preferences (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id          UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE UNIQUE,
  preferred_tone        TEXT DEFAULT 'helpful_expert',
  cta_style             TEXT DEFAULT 'free_inspection',
  outreach_style        TEXT DEFAULT 'balanced',
  communication_style   TEXT DEFAULT 'helpful_direct',
  dm_style              TEXT DEFAULT 'conversational',
  response_personality  TEXT DEFAULT 'empathetic_pro',
  signal_priorities     TEXT[],   -- e.g. ['recommendation_requests',
'competitor_complaints']
  excluded_keywords     TEXT[],
  excluded_industries   TEXT[],
  preferred_lead_types  TEXT[],   -- e.g. ['homeowners', 'property_managers']
  high_intent_only      BOOLEAN DEFAULT false,
  scoring_sensitivity   INTEGER DEFAULT 50,  -- 0=balanced, 100=recommended only
  updated_at            TIMESTAMPTZ DEFAULT now()
);
```

Table: notification_preferences

```
CREATE TABLE notification_preferences (
  id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id            UUID NOT NULL REFERENCES workspaces(id) UNIQUE,
  urgent_alerts_email     BOOLEAN DEFAULT true,
  urgent_alerts_sms       BOOLEAN DEFAULT false,
  urgent_alerts_inapp     BOOLEAN DEFAULT true,
  daily_digest_email      BOOLEAN DEFAULT true,
  daily_digest_inapp      BOOLEAN DEFAULT true,
  high_conversion_email   BOOLEAN DEFAULT true,
  high_conversion_inapp   BOOLEAN DEFAULT true,
  new_dm_opps_email       BOOLEAN DEFAULT false,
  new_dm_opps_inapp       BOOLEAN DEFAULT true,
  weekly_insights_email   BOOLEAN DEFAULT true,
  weekly_insights_inapp   BOOLEAN DEFAULT true
);
```

Table: message_tracking

Every time a user copies a comment or DM, marks it sent, marks it replied, or marks a link clicked, one record is written here. This is the V1 performance tracking layer. The airtable_message_id links back to the messages table in Airtable.

```
CREATE TABLE message_tracking (
```

```
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id          UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
  user_id               UUID NOT NULL REFERENCES auth.users(id),
  airtable_message_id   TEXT NOT NULL,       -- links to Airtable messages.message_id
  airtable_opp_id       TEXT NOT NULL,       -- links to Airtable
opportunities.opportunity_id
  message_type          TEXT NOT NULL,        -- comment | dm
  angle_type            TEXT,                 -- balanced | friendly | direct |
soft_assist | consultative | direct_offer
  offer_used            TEXT,
  platform              TEXT,
  action                TEXT NOT NULL,        -- copied | sent | replied | clicked |
regenerated
  action_at             TIMESTAMPTZ DEFAULT now()
);
```

Table: performance_summary

Aggregated performance view per workspace. Updated by Make.com weekly (Scenario 8 syncs insight data here) and also updated in real-time by frontend as users track actions. Used to power the Insights page KPI cards.

```
CREATE TABLE performance_summary (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id        UUID NOT NULL REFERENCES workspaces(id) UNIQUE,
  total_copied        INTEGER DEFAULT 0,
  total_sent          INTEGER DEFAULT 0,
  total_replied       INTEGER DEFAULT 0,
  total_clicked       INTEGER DEFAULT 0,
  reply_rate          NUMERIC(5,2) DEFAULT 0,
  click_rate          NUMERIC(5,2) DEFAULT 0,
  best_angle          TEXT,
  best_offer          TEXT,
  best_tone           TEXT,
  best_hook           TEXT,
  best_cta            TEXT,
  ai_insight_summary  TEXT,        -- synced from Airtable insights table by Make.com
  updated_at          TIMESTAMPTZ DEFAULT now()
);
```

Table: dashboard_cache

Lightweight cache table that Make.com writes to after each pipeline run. The Dashboard page reads from here instead of calling Airtable on every page load. TTL: 30 minutes. Invalidated on new scrape run.

```
CREATE TABLE dashboard_cache (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id          UUID NOT NULL REFERENCES workspaces(id) UNIQUE,
  total_opps_today      INTEGER DEFAULT 0,
  high_intent_count     INTEGER DEFAULT 0,
  urgent_count          INTEGER DEFAULT 0,
  recommendation_count  INTEGER DEFAULT 0,
```

```
  competitor_dissatisfaction_count INTEGER DEFAULT 0,
  price_shopping_count    INTEGER DEFAULT 0,
  other_count             INTEGER DEFAULT 0,
  opps_vs_yesterday_pct   NUMERIC(5,1),
  platform_breakdown      JSONB,            -- {"google":14,"reddit":9,"facebook":8}
  top_opportunity_feed    JSONB,            -- array of top 5 opps for dashboard feed
  best_angle              TEXT,
  best_hook               TEXT,
  best_cta                TEXT,
  best_tone               TEXT,
  reply_rate_uplift       NUMERIC(5,1),
  last_pipeline_run_at    TIMESTAMPTZ,
  cache_expires_at        TIMESTAMPTZ,
  updated_at              TIMESTAMPTZ DEFAULT now()
);
```

Table: notifications

```
CREATE TABLE notifications (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id  UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
  user_id       UUID NOT NULL REFERENCES auth.users(id),
  type          TEXT NOT NULL,   -- urgent_opportunity | new_insights | system |
onboarding
  title         TEXT NOT NULL,
  body          TEXT,
  link          TEXT,
  read          BOOLEAN DEFAULT false,
  created_at    TIMESTAMPTZ DEFAULT now()
);
```

Row-Level Security (RLS) Policies — Required

> **RLS Must Be Configured Before Any User Data Is Exposed**
>
> Enable RLS on every table listed below.
> Test with two separate user accounts to confirm they cannot read each other's data.
> A misconfigured RLS policy is a critical security vulnerability.
> Use the Supabase service role key ONLY in server-side code — never in the browser.

```
-- Enable RLS on all tables
ALTER TABLE workspaces ENABLE ROW LEVEL SECURITY;
ALTER TABLE workspace_members ENABLE ROW LEVEL SECURITY;
ALTER TABLE ai_preferences ENABLE ROW LEVEL SECURITY;
ALTER TABLE notification_preferences ENABLE ROW LEVEL SECURITY;
ALTER TABLE message_tracking ENABLE ROW LEVEL SECURITY;
ALTER TABLE performance_summary ENABLE ROW LEVEL SECURITY;
ALTER TABLE dashboard_cache ENABLE ROW LEVEL SECURITY;
ALTER TABLE notifications ENABLE ROW LEVEL SECURITY;
```

```
-- workspaces: users can only see workspaces they own or are members of
CREATE POLICY "workspace_access" ON workspaces
  FOR ALL USING (
    owner_id = auth.uid() OR
    id IN (SELECT workspace_id FROM workspace_members WHERE user_id = auth.uid() AND
accepted = true)
  );

-- workspace_members: only members of the workspace can see its membership
CREATE POLICY "member_access" ON workspace_members
  FOR ALL USING (
    workspace_id IN (SELECT id FROM workspaces WHERE owner_id = auth.uid()) OR
    user_id = auth.uid()
  );

-- All other tables: access scoped to workspace the user belongs to
CREATE POLICY "workspace_scoped" ON message_tracking
  FOR ALL USING (workspace_id IN (SELECT id FROM workspaces WHERE owner_id = auth.uid()));

-- (Repeat similar policy for: ai_preferences, notification_preferences,
--  performance_summary, dashboard_cache, notifications)

-- dashboard_cache: Make.com writes via service role key (bypasses RLS)
-- performance_summary: Make.com writes via service role key
-- These tables also need user-read policies so frontend can read them
```

SECTION 3 API ARCHITECTURE

API Route Map

All API routes live under /app/api/. Server Components can call Supabase and Airtable directly. Client Components go through API routes for Airtable reads (to keep the Airtable API key server-side only).

Route	Method	Auth Required	Purpose
/api/airtable/opportunities	GET	Yes	Fetch opportunities from Airtable filtered by workspace client_id
/api/airtable/opportunities/[id]	GET	Yes	Fetch single opportunity with comment + DM angles
/api/airtable/comments	GET	Yes	Fetch comment-eligible opportunities with generated angles
/api/airtable/dms	GET	Yes	Fetch DM-eligible opportunities with generated DM angles
/api/airtable/insights	GET	Yes	Fetch insights records for this workspace
/api/tracking	POST	Yes	Log a copy/send/reply/click action to message_tracking
/api/tracking/performance	GET	Yes	Fetch aggregated performance for Insights page
/api/webhooks/stripe	POST	Stripe sig	Receive Stripe events (payment, cancellation, upgrade)
/api/webhooks/make	POST	HMAC secret	Receive Make.com events (new opps, pipeline complete)
/api/stripe/checkout	POST	Yes	Create Stripe checkout session
/api/stripe/portal	POST	Yes	Create Stripe customer portal session
/api/onboarding/complete	POST	Yes	Trigger Make.com Scenario 10 webhook after onboarding

Airtable API Client (Server-Side Only)

The Airtable API key must NEVER be exposed to the browser. All Airtable reads happen inside Next.js API routes or Server Components. The frontend makes a fetch to /api/airtable/* which then calls Airtable server-side.

```
// lib/airtable/client.ts
import Airtable from 'airtable';

const base = new Airtable({
  apiKey: process.env.AIRTABLE_API_KEY  // Server-only env var, no NEXT_PUBLIC_ prefix
}).base(process.env.AIRTABLE_BASE_ID!);
```

```
export { base };

// lib/airtable/queries.ts
import { base } from './client';

export async function getOpportunities(clientId: string, filters?: OpportunityFilters) {
  const filterFormula = buildFilterFormula(clientId, filters);
  const records = await base('opportunities')
    .select({
      filterByFormula: filterFormula,
      sort: [{ field: 'opportunity_score', direction: 'desc' }],
      maxRecords: 50
    })
    .all();
  return records.map(r => mapOpportunityRecord(r));
}

function buildFilterFormula(clientId: string, filters?: OpportunityFilters): string {
  const conditions = [`{client_id} = "${clientId}"`];
  if (filters?.platform && filters.platform !== 'all') {
    conditions.push(`{platform} = "${filters.platform}"`);
  }
  if (filters?.urgency && filters.urgency !== 'all') {
    conditions.push(`{urgency} = "${filters.urgency}"`);
  }
  if (filters?.intentLevel && filters.intentLevel !== 'all') {
    conditions.push(`{intent_level} = "${filters.intentLevel}"`);
  }
  if (filters?.signalType && filters.signalType !== 'all') {
    conditions.push(`{signal_type} = "${filters.signalType}"`);
  }
  if (filters?.minScore) {
    conditions.push(`{opportunity_score} >= ${filters.minScore}`);
  }
  return `AND(${conditions.join(', ')})`;
}
```

Tracking API Route

```
// app/api/tracking/route.ts
import { createServerClient } from '@/lib/supabase/server';
import { NextRequest, NextResponse } from 'next/server';

export async function POST(req: NextRequest) {
  const supabase = createServerClient();
  const { data: { session } } = await supabase.auth.getSession();
  if (!session) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  const body = await req.json();
  const { airtable_message_id, airtable_opp_id, message_type,
          angle_type, offer_used, platform, action } = body;

  // Get the user's workspace
  const { data: workspace } = await supabase
```

```
    .from('workspaces')
    .select('id')
    .eq('owner_id', session.user.id)
    .single();

  // Insert tracking record
  await supabase.from('message_tracking').insert({
    workspace_id: workspace.id,
    user_id: session.user.id,
    airtable_message_id, airtable_opp_id, message_type,
    angle_type, offer_used, platform, action
  });

  // Update performance_summary counts
  await updatePerformanceSummary(supabase, workspace.id, action);

  return NextResponse.json({ success: true });
}
```

Make.com Webhook Handler

```
// app/api/webhooks/make/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { createServiceClient } from '@/lib/supabase/server';
import { verifyWebhookSecret } from '@/lib/utils';

export async function POST(req: NextRequest) {
  // Verify HMAC secret to confirm request is from Make.com
  const signature = req.headers.get('x-necty-signature');
  const body = await req.text();
  if (!verifyWebhookSecret(body, signature)) {
    return NextResponse.json({ error: 'Invalid signature' }, { status: 401 });
  }

  const payload = JSON.parse(body);
  const supabase = createServiceClient(); // Service role — bypasses RLS

  switch (payload.event) {
    case 'pipeline_complete':
      // Update dashboard_cache for this workspace
      await supabase.from('dashboard_cache').upsert({
        workspace_id: payload.supabase_workspace_id,
        ...payload.cache_data,
        cache_expires_at: new Date(Date.now() + 30 * 60 * 1000).toISOString()
      });
      break;
    case 'new_urgent_opportunity':
      // Create notification
      await supabase.from('notifications').insert({
        workspace_id: payload.supabase_workspace_id,
        user_id: payload.owner_id,
        type: 'urgent_opportunity',
        title: 'New urgent opportunity detected',
        body: payload.opportunity_title,
```

```
        link: '/opportunities'
      });
      break;
    case 'weekly_insights_ready':
      // Sync insights summary to performance_summary
      await supabase.from('performance_summary').upsert({
        workspace_id: payload.supabase_workspace_id,
        ...payload.performance_data
      });
      break;
  }

  return NextResponse.json({ received: true });
}
```

Environment Variables

```
# .env.local — never commit this file

# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://[project].supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=[anon-key]      # Safe for browser
SUPABASE_SERVICE_ROLE_KEY=[service-key]        # Server-only, never NEXT_PUBLIC_

# Airtable — SERVER ONLY, never expose to browser
AIRTABLE_API_KEY=[pat-key]
AIRTABLE_BASE_ID=[base-id]

# Stripe
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=[pk_live_...]
STRIPE_SECRET_KEY=[sk_live_...]               # Server-only
STRIPE_WEBHOOK_SECRET=[whsec_...]             # Server-only
STRIPE_PRO_PRICE_ID=[price_...]

# Make.com webhook
MAKE_WEBHOOK_SECRET=[random-32-char-hex]      # For HMAC verification
MAKE_ONBOARDING_WEBHOOK_URL=[https://hook.make.com/...]

# App
NEXT_PUBLIC_APP_URL=https://app.necty.io

# Email (Resend)
RESEND_API_KEY=[re_...]
```

SECTION 4 AUTHENTICATION & ONBOARDING FLOW

Auth Architecture

NECTY uses Supabase Auth with email + password. All routes under /(dashboard)/ are protected by Next.js middleware. Unauthenticated users are redirected to /login. The middleware refreshes the Supabase session on every request.

```
// middleware.ts
import { createMiddlewareClient } from '@supabase/auth-helpers-nextjs';
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export async function middleware(req: NextRequest) {
  const res = NextResponse.next();
  const supabase = createMiddlewareClient({ req, res });
  const { data: { session } } = await supabase.auth.getSession();

  const protectedRoutes = ['/dashboard', '/opportunities', '/comments',
    '/dms', '/insights', '/sources', '/settings', '/help'];

  const isProtected = protectedRoutes.some(r => req.nextUrl.pathname.startsWith(r));

  if (isProtected && !session) {
    return NextResponse.redirect(new URL('/login', req.url));
  }

  // Redirect logged-in user away from auth pages
  if (session && ['login', '/signup'].includes(req.nextUrl.pathname)) {
    return NextResponse.redirect(new URL('/dashboard', req.url));
  }

  // Check onboarding completion
  if (session && req.nextUrl.pathname !== '/onboarding') {
    // Check if workspace exists for this user
    // If not, redirect to /onboarding
  }

  return res;
}

export const config = { matcher: ['/((?!_next/static|_next/image|favicon.ico).*)'] };
```

Onboarding Flow — Step by Step

After signup, users without a workspace are routed to /onboarding. This is a multi-step form. On completion, the frontend calls /api/onboarding/complete which triggers Make.com Scenario 10.

Step	Fields Collected	Supabase Write	UX Notes
1. Welcome	None	None	Explain what NECTY does in 2 sentences. CTA: Get Started
2. Business Info	business_name, industry (hybrid dropdown), city, state	workspaces record created	Industry: 30-option searchable dropdown + free text escape hatch
3. Service Details	service_area_mi, description, specialties, booking_link, website_url, phone	workspaces updated	Booking link is critical — explain it is used in generated messages
4. AI Preferences	preferred_tone, cta_style, outreach_style	ai_preferences created	Show live preview of how AI will sound with selected tone
5. Platform Setup	competitors_to_monitor, signal_priorities, excluded_keywords	ai_preferences updated	Tell NECTY what to look for and what to ignore
6. Plan Selection	plan selection	Stripe checkout opened	After payment, Stripe webhook updates workspaces.plan
7. Go Live	None	None	Trigger Make.com Scenario 10, redirect to Dashboard

```
// app/api/onboarding/complete/route.ts
// Called after all onboarding steps are complete.
// Triggers Make.com Scenario 10 which creates Airtable records
// and fires the first scrape run.

export async function POST(req: NextRequest) {
  const supabase = createServerClient();
  const { data: { session } } = await supabase.auth.getSession();

  // Get workspace + preferences
  const { data: workspace } = await supabase
    .from('workspaces').select('*, ai_preferences(*)').eq('owner_id',
session.user.id).single();

  // Fire Make.com Scenario 10 webhook
  await fetch(process.env.MAKE_ONBOARDING_WEBHOOK_URL!, {
```

```
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      supabase_workspace_id: workspace.id,
      business_name: workspace.business_name,
      industry: workspace.industry,
      custom_industry: workspace.custom_industry,
      city: workspace.city,
      state: workspace.state,
      booking_link: workspace.booking_link,
      preferred_tone: workspace.ai_preferences?.preferred_tone,
      signal_priorities: workspace.ai_preferences?.signal_priorities,
    })
  });

  return NextResponse.json({ success: true });
}
```

SECTION 5 PAGE-BY-PAGE FRONTEND SPECIFICATIONS

Each page specification includes: purpose, primary UX goal, required data, components, interaction logic, state, filtering, responsiveness, and all UI states. Build each page exactly as specified. Navigation is a fixed left sidebar on desktop, bottom tab bar on mobile.

Global Layout — /(dashboard)/layout.tsx

All authenticated pages share this layout. It renders the sidebar navigation, top bar, and notification bell. The sidebar is collapsible on desktop and becomes a bottom nav on mobile.

- Sidebar width: 200px expanded, 64px collapsed. Persists collapse state in localStorage.
- Nav items: Dashboard, Opportunities, Comments, DMs (with "New" badge when new DM opps exist), Insights, Sources. Below divider: Settings, Help.
- Active state: orange text + orange left border on the active nav item.
- Top bar: shows workspace switcher (city + industry), current date, notifications bell with unread count badge.
- User profile section at bottom of sidebar: avatar, name, plan badge, Manage Plan button.
- Mobile: sidebar becomes a bottom tab bar with icon + label for: Dashboard, Opportunities, Comments, DMs, More (opens drawer with Insights, Sources, Settings, Help).

PAGE: Dashboard

Route: /dashboard | Primary data: dashboard_cache (Supabase), Airtable top_opportunity_feed

Purpose

The daily command center. The user lands here every session. It must immediately answer: "What happened today? Where should I focus right now?"

Primary UX Goal

Show the most important signal in under 3 seconds: how many high-intent opportunities exist today, what messaging is working, and surface the live feed so the user can take action without clicking away.

Required Data Sources

- dashboard_cache (Supabase): opportunity counts, platform breakdown, best angle/hook/CTA, reply_rate_uplift
- top_opportunity_feed (JSONB from dashboard_cache): top 5 opportunities with comment + DM angle previews
- performance_summary (Supabase): Messaging Intelligence section data

Component Hierarchy

```
DashboardPage
├── DashboardHeader (greeting + date)
├── TodaysOpportunitiesCard
```

```
│   ├── TotalCount (large number)
│   ├── SignalTypeBreakdown (list with counts + %)
│   └── ViewOpportunitiesButton
├── BestMessagingAngleCard
│   ├── BestHook (quoted text)
│   ├── ReplyRateUplift (green % badge)
│   ├── BestCTAExample
│   └── TopToneStyle
├── HighIntentOpportunityFeed
│   ├── FeedHeader (Live badge + auto-refresh toggle)
│   ├── FeedTable (5 rows)
│   │   └── FeedRow (source icon, preview, location, intent badge,
│   │               signal type, best comment angle, best DM angle, Open button)
│   └── ViewAllButton
├── MessagingIntelligenceSection
│   ├── BestHookThisWeek
│   ├── BestPerformingCTA
│   ├── TopToneStyle
│   └── BestConvertingTypes
└── PlatformActivityToday
    └── PlatformCard × n (icon, count, vs yesterday %)
```

Interaction Logic

- View Opportunities button: navigates to /opportunities
- Open Opportunity button in feed row: navigates to /opportunities with that opportunity selected (pass ?id=OPP-xxx in URL)
- Auto-refresh toggle: when ON, polls /api/airtable/opportunities every 90 seconds and updates feed if new items appear
- Notification bell: opens a dropdown with recent unread notifications. Mark all read clears the badge.

State

- dashboardData: loaded from dashboard_cache via Supabase query on mount. Cache TTL 30 min — if cache_expires_at has passed, show a "Refreshing..." indicator while Make.com pipeline is running.
- opportunityFeed: parsed from dashboard_cache.top_opportunity_feed JSONB field
- autoRefresh: boolean in Zustand store, persisted in localStorage

Loading State

Skeleton cards for each section. Show animated pulse on the count numbers. Never show a blank page.

Empty State

If no opportunities exist yet (new workspace, first run pending): show "NECTY is scanning your market. Your first opportunities will appear within 30 minutes." with a progress indicator and a "Check Sources" button linking to /sources.

Error State

If dashboard_cache query fails: show last-known data with a "Data may be outdated" badge. Never show a full error screen on the dashboard.

Responsiveness

Mobile: stack all cards vertically. TodaysOpportunitiesCard at top. SignalTypeBreakdown becomes a 2-col grid. FeedTable collapses to cards. Messaging Intelligence becomes horizontal scroll. PlatformActivity becomes 2-col grid.

PAGE: Opportunities

Route: /opportunities | Primary data: Airtable opportunities table

Purpose

The live opportunity feed. Every high-intent conversation NECTY has found, scored, and queued for action. This is where users decide what to respond to and get both a comment and DM angle in one view.

Primary UX Goal

Fast scanning. The user should be able to look at the page, identify the highest-priority opportunity, see the suggested message angles, and take action in under 60 seconds.

Required Data

- Airtable: opportunities table filtered by client_id, sorted by opportunity_score DESC, with linked messages records
- Supabase: message_tracking for this workspace (to show "already responded" state)

Component Hierarchy

```
OpportunitiesPage
├── PageHeader (title, subtitle, search, Filters button, sort)
├── KPIBar (Total, High-Intent, Urgent, Recommendation, Competitor, Price Shopping)
├── FilterBar (Platform, Location, Urgency, Intent Type, Conversion Likelihood,
│          Keyword/Topic, More Filters)
├── LiveFeedIndicator (green dot + "New conversations detected X seconds ago" +
Auto-refresh toggle)
├── OpportunityList
│   └── OpportunityRow × n
│       ├── SourceIcon (platform logo)
│       ├── ConversationPreview (text, author, time ago, location)
│       ├── IntentBadge (High/Medium/Low)
│       ├── SignalTypeBadge
│       ├── SuggestedApproach (text)
│       ├── BestCommentAngle (preview text)
│       ├── BestDMAngle (preview text)
│       ├── AIScore (circle badge)
│       └── ChevronRight (expands to selected state)
└── SelectedOpportunityPanel (right sidebar, shown when row is clicked)
    ├── PanelHeader (title, × close)
    ├── OpportunityMeta (platform, author, time, location, intent, signal type)
    ├── TabBar (Best Comment Angles | Best DM Angles)
    ├── CommentAngleCards × 3 (angle name, text, Copy button, Regenerate icon)
    ├── DMAngleCards × 3
    ├── AIConfidenceBar
    └── QuickActions (Save, Open Thread, Copy Comment, Copy DM, Regenerate All, Mark
Responded, Add Notes)
```

Selected Opportunity Panel

When a user clicks a row, the right-side panel slides in. It shows the full opportunity context and all generated message angles. This panel does NOT navigate away from the list — the list stays visible on the left.

- On desktop: panel is 380px wide, appears alongside the list. List shrinks.
- On tablet: panel appears as a full-width overlay from the bottom (like a sheet).
- On mobile: panel takes full screen with a back button.

Copy Action

```
// When user clicks Copy on a comment or DM angle:
1. navigator.clipboard.writeText(message_text)
2. Show "Copied!" toast notification for 2 seconds
3. POST to /api/tracking with: {
     airtable_message_id, airtable_opp_id, message_type: "comment" | "dm",
     angle_type, offer_used, platform, action: "copied"
   }
4. Button shows checkmark state for 3 seconds, then resets
5. Row in opportunity list shows a subtle "Copied" indicator
```

Filter State

All active filters are stored in Zustand filterStore and persisted in sessionStorage. Navigating back to /opportunities restores the last filter state. Filter values are passed as query params to /api/airtable/opportunities on each change.

Search

Cmd+K opens the search bar. Search queries the conversation preview text and location. This is a client-side filter over already-loaded opportunities (no new API call on each keystroke). Debounced at 300ms.

Loading State

Show 8 skeleton rows with animated pulse. KPIBar shows dashes.

Empty State

If filters return no results: "No opportunities match your filters." with Clear Filters button. If no opportunities at all: "NECTY hasn't found opportunities yet. Check your Sources to make sure monitoring is active." with Sources button.

PAGE: Comments

Route: /comments | Primary data: Airtable opportunities (comment_viability = TRUE) + messages

Purpose

Shows every opportunity where leaving a public comment makes sense. For each, shows 3 AI-generated comment angles with reply likelihood scores. Also shows an Engagement Strategy panel on the right with today's best performing hooks and approach.

Primary UX Goal

Enable the user to scan real public conversations, pick the best comment angle, copy it, and post it — in under 90 seconds per opportunity.

Page-Specific KPI Cards

- Public Opportunities Found Today
- Urgent Conversations (needs response now)
- Recommendation Requests
- Competitor Dissatisfaction
- Top Converting Angle Today (right-aligned callout with % uplift)

Component Hierarchy

```
CommentsPage
├── PageHeader + search + Filters + Sort
├── KPIBar (4 cards + Top Angle callout)
├── FilterBar (Platform, Urgency, Signal Type, Intent, Likelihood, Pain Point, Sentiment,
Score)
├── LiveFeedIndicator
├── CommentOpportunityList
│   └── CommentOpportunityCard × n
│       ├── PlatformIcon + AIScore + IntentLevel
│       ├── PostPreview (text, author, time, location)
│       ├── SignalTypeBadge + IntentBadge + UrgencyBadge
│       ├── WhyNECTYFlaggedThis (text)
│       ├── AICommentAngles (3 angle cards)
│       │   └── CommentAngleCard
│       │       ├── AngleName + AngleTypeBadge
│       │       ├── CommentText
│       │       ├── ReplyLikelihoodBar
│       │       ├── CopyButton + RegenerateIcon
│       │       └── OpenThreadLink
│       └── ExpandCollapseToggle
└── RightPanel: EngagementStrategy
    ├── SuggestedStrategy (text)
    ├── BestHookToday (quoted, orange)
    ├── TopConvertingTone
    ├── RecommendedCTAStyle
    ├── WhyThisOpportunityMatters
    └── InsightsSummary (hooks, reply-rate CTAs, performing tones, top signal)
```

Comment Angle Cards

Each opportunity shows 3 comment angle cards in a horizontal row on desktop, stacked on mobile. Cards are: Helpful Expert, Friendly Local Proof, Direct Offer — matching the wireframe. Each card shows the full comment text, reply likelihood as a percentage with a colored bar, a Copy button, and a Regenerate icon.

Reply Likelihood Display

Reply likelihood is a number from the Airtable messages record (ai_confidence_score field mapped to reply likelihood in the UI). Display as: colored progress bar (green > 70%, yellow 50–70%, orange < 50%) + percentage text.

Copy Interaction

Same copy flow as Opportunities page: clipboard write → "Copied!" toast → POST to /api/tracking with action: "copied", message_type: "comment". The specific angle_type is passed so performance data can be tracked per angle.

Filtering

- Platform: All / Google / Reddit / Facebook / TikTok / Nextdoor / Instagram
- Urgency: All / High / Medium / Low
- Signal Type: All / Recommendation Request / Competitor Dissatisfaction / Urgent Need / Price Shopping / General Inquiry
- Intent: All / High / Medium / Low
- Likelihood: All / High (70%+) / Medium / Low
- Score: slider from 0–100, default 70+

PAGE: DMs

Route: /dms | Primary data: Airtable opportunities (dm_viability = TRUE) + messages

Purpose

Shows every opportunity where sending a private DM is viable (dm_viability = TRUE in Airtable). For each, shows 3 AI-generated DM variations with reply likelihood, conversion likelihood, and a "Most Effective" badge on the recommended angle. Includes a DM Conversion Strategy panel and AI Coaching Insight.

Primary UX Goal

Help the user identify the right person to DM, pick the most persuasive message, and send it quickly. The DM page should feel more personal and deliberate than the Comments page.

Page-Specific KPI Cards

- High-Intent DM Opportunities (count)
- Urgent Outreach (needs DM now)
- Top DM Angle Today (text label)
- Reply Rate Uplift (% vs average)
- Best Performing CTA (quoted text)

Component Hierarchy

```
DMsPage
├── PageHeader ("Your next clients are in your DMs, Karyn") + filters + sort
├── KPIBar (5 cards)
├── SearchBar + FilterBar (Platform, Urgency, Location, Signal Type, Intent, Conversion Likelihood)
├── DMOpportunityList
│   └── DMOpportunityCard × n
│       ├── PlatformIcon + PostPreview + Author + Time + Location
│       ├── SignalTypeBadge + UrgencyBadge + IntentLevelBadge
│       ├── WhyNECTYFlaggedThis
│       ├── OpportunityScore
│       ├── SuggestedOutreachStrategy (label + description)
│       ├── AIDMAngles (3 DM cards)
│       │   └── DMAngleCard
│       │       ├── AngleNumber + AngleName + MostEffectiveBadge (conditional)
│       │       ├── DMText
│       │       ├── ToneLabel
│       │       ├── ReplyLikelihood + ConversionLikelihood (with trend arrows)
```

```
│       │       └── CopyIcon + RegenerateIcon
│       └── MessageThisLeadButton (opens DM composer)
└── RightPanel: DMConversionStrategy
    ├── BestDMStrategyToday
    ├── TopConvertingTone
    ├── HighestConvertingCTA
    ├── ReplyRateUplift
    ├── ConversionRateUplift
    ├── AICoachingInsight (quoted card)
    ├── RecommendedFollowUp
    ├── UrgencyRecommendation (Act Now badge)
    └── DMIntelligenceOverview (best hooks, CTAs, tones, pain points)
```

"Message This Lead" Button

This button opens the DM composer. In V1, this is a pre-filled text panel that shows the DM text with a "Copy to Clipboard" button. It does NOT auto-send. The user copies and sends manually from the platform. The panel also shows: "Open [Platform] Profile" link to the original post.

"Most Effective" Badge Logic

The Airtable messages record includes a field ranking DM variations by predicted performance (based on signal_type, urgency, and platform). The highest-ranked DM gets the "Most Effective" badge in orange. This is passed from Airtable — the frontend just renders it.

"New" Badge on Nav Item

The DMs nav item shows a "New" badge when there are DM opportunities the user has not yet viewed in the current session. Store viewed opportunity IDs in sessionStorage. Clear the badge once the DMs page is visited.

PAGE: Insights

Route: /insights | Primary data: Supabase performance_summary + Airtable insights

Purpose

The learning layer. Shows which messaging approaches are driving replies and conversions. Surfaces AI recommendations on what to use more and what to stop doing. Updated weekly by Make.com Scenario 8.

Primary UX Goal

In one page, the user should understand: what's working, what's not, and what to do differently this week. This page should feel like a smart marketing analyst briefing, not a data dump.

AI Messaging Intelligence Summary Bar

Top of page. Shows 5 KPI callouts in one horizontal band: Best Performing Angle, Reply Rate Uplift, Top CTA, Top Tone/Style, AI Insight (1-sentence narrative).

Component Hierarchy

```
InsightsPage
```

```
├── PageHeader + filters (date range, city/industry)
├── IntelligenceSummaryBar (5 KPI callouts)
├── TopPerformingHooksTable
│   ├── HookRow × n (hook text, reply rate, conversion rate, trend sparkline, signal type
badges)
│   └── ViewAllHooks link
├── BestConvertingOffersGrid
│   └── OfferCard × 5 (ranked 1–5, offer name, conversion %, top platforms, tone)
├── PainPointIntelligenceTable
│   ├── PainPointRow (pain point, conversion uplift %)
│   └── TrendingConversationThemesColumn (theme, trend sparkline)
├── MessageStylePerformanceTable
│   └── StyleRow × n (style name, reply rate bar, conversion rate bar, engagement rate
bar)
├── PlatformInsightsGrid
│   └── PlatformCard × 5 (platform logo, reply rate, best messaging, top signals)
└── RightPanel: AIRecommendations
    ├── TopMessagingToLeanInto (bulleted list)
    ├── HooksToTestThisWeek
    ├── StylesUnderperforming (red × list)
    └── EmergingTrends
```

Data Flow

performance_summary (Supabase) provides: best_angle, best_offer, best_tone, best_hook, best_cta, reply_rate, click_rate, ai_insight_summary. Airtable insights table provides: full hook rankings, offer breakdowns, platform intelligence, style performance. Frontend calls both and merges.

Empty State

"NECTY is still collecting data. Insights become available after your first week of activity. Check back soon." Show a subtle progress bar indicating data collection in progress.

PAGE: Sources

Route: /sources | Primary data: Supabase workspace (platforms config) + Airtable sources

Purpose

Shows which platforms NECTY is monitoring and their performance stats. Lets users add/remove sources, configure search terms, and fine-tune AI discovery preferences.

Component Hierarchy

```
SourcesPage
├── PageHeader + filters (city, industry)
├── KPIBar (Connected Sources, Opportunities Today, Top Performer, Most Active Source,
Monitoring Status)
├── ConnectedSourcesGrid
│   └── SourceCard × n (platform logo, Connected badge, opportunities this week,
│       response rate, Monitoring toggle, last sync time, Pause toggle,
│       Configure button, View Opportunities button)
│   └── AddSourceButton (opens modal)
├── SearchConfigurationPanel
│   ├── KeywordsToMonitor (tag input)
```

```
│   ├── ServiceCategories (checklist)
│   ├── LocationTargeting (city tags + radius dropdown)
│   ├── SignalTypesToTrack (checklist)
│   └── PrioritySettings (radio group)
│   └── SaveConfigurationButton
├── AIDiscoveryPreferences
│   ├── PreferredMessagingTone (radio)
│   ├── PreferredLeadTypes (multi-select)
│   ├── IndustriesToAvoid (tag input)
│   ├── ExcludedKeywords (tag input)
│   └── OpportunityScoringSlider
└── SourcePerformanceSnapshot
    ├── HighestIntentLeads
    ├── BestConvertingSource
    ├── FastestGrowingSource
    └── HighestReplyRate
```

Write Logic

When the user saves Search Configuration or AI Discovery Preferences, the frontend writes to Supabase (ai_preferences table). Make.com reads these on the next pipeline run via the airtable_client_id link. No direct write to Airtable from the frontend.

Platform Cards — Pause Monitoring Toggle

Toggling a platform off updates the sources table in Airtable via Make.com — the frontend calls /api/webhooks/make equivalent with a sources_update event, which triggers Make.com to set the source status to Paused.

PAGE: Settings

Route: /settings | Primary data: Supabase workspaces, ai_preferences, notification_preferences

Purpose

The business profile and AI configuration hub. Users control everything about how NECTY finds and generates messages for them.

Component Hierarchy

```
SettingsPage
├── WorkspaceHeader (logo, business name, Verified badge, KPI badges:
│   industry, service area, connected platforms, AI optimization score, onboarding %)
├── EditBusinessProfile + RunAIOptimizationCheck buttons
├── BusinessProfileSection
│   ├── BusinessNameInput, IndustryCategoryInput
│   ├── ServiceAreasSelect, YearsInBusinessInput
│   ├── BusinessDescriptionTextarea
│   ├── SpecialitiesTagInput
│   ├── WebsiteURLInput, PhoneInput, BookingLinkInput
│   └── SaveChangesButton
│   └── RightPanel: AISuggestions (3 cards with Apply buttons)
├── AIMessagingPreferencesSection
│   ├── ToneDropdown, CTAStyleDropdown, OutreachStyleDropdown
```

```
│   ├── CommunicationStyleDropdown, DMStyleDropdown, ResponsePersonalityDropdown
│   ├── StylePreviewPanel (shows side-by-side: selected vs more direct)
│   └── TestDifferentCombinationsButton
├── OpportunityTargetingSection
│   ├── PreferredLeadTypesToggleButtons
│   ├── SignalTypesToPrioritizeToggles
│   ├── ExcludedKeywordsTagInput
│   ├── UrgencyPreferencesDropdown
│   ├── HighIntentOnlyToggle
│   ├── ExcludedIndustriesTagInput
│   └── GeographicTargetingDropdown
├── ConnectedAccountsSection (same as Sources page cards, condensed)
├── NotificationSettingsSection
│   └── NotificationMatrix (notification type × email/sms/in-app toggles)
├── TeamAndWorkspaceSection
│   ├── WorkspaceNameInput
│   ├── TeamMembersTable (name, role, Full Access/Editor/Viewer dropdown)
│   └── InviteMemberButton (opens modal: email input + role select)
└── AIPerformanceHealthSection
    ├── MessagingOptimizationScore (score + label)
    ├── ResponsePerformanceTrendSparkline
    ├── AIRecommendationQuality %
    └── AISuggestionsToImproveCards
```

Save Behavior

Business Profile section: Save Changes button is explicit. All other sections auto-save on change with a "Saved" indicator. All writes go to Supabase. Changes to ai_preferences are picked up by Make.com on the next pipeline run.

AI Optimization Check

"Run AI Optimization Check" button triggers a call to /api/airtable/insights to fetch the latest AI suggestion cards and refresh the AI Suggestions panel in real time.

PAGE: Help

Route: /help | Primary data: Static content + Supabase onboarding progress

Purpose

The Help page is also the onboarding support hub. It shows progress on onboarding completion, gives access to quick start guides, AI messaging playbooks, troubleshooting, video resources, and live support.

Component Hierarchy

```
HelpPage
├── PageHeader
├── MainContent (left/center)
│   ├── AISearchBar ("Ask anything...")
│   ├── TryAsking chips (3 example queries)
│   ├── SupportActionBar (Watch Quick Start, Book Call, Contact Support, Messaging Tips)
│   ├── QuickStartGuides (6 guide cards with level badge + time estimate)
│   ├── AIMessagingPlaybooks (6 playbook cards: Helpful Expert, Empathy-Based, etc.)
```

```
│   ├── TroubleshootingSupportCenter (6 issue cards)
│   └── ContactSupportOptions (Live Chat, Email, Book Call, Community)
├── OnboardingProgress (right panel)
│   ├── ProgressCircle (% complete)
│   ├── ChecklistItems (Connect Sources, Write First Comment, etc.)
│   └── ViewOnboardingGuideLink
└── AIRecommendations (right panel, below progress)
    ├── NextBestAction card
    ├── SetupImprovements card
    ├── MessagingOpportunity card
    └── YouMayBeMissing card
```

AI Search Bar

Searches a static knowledge base (can be implemented as a hardcoded FAQ object in V1 or a simple vector search if Supabase pgvector is available). Returns contextual answers about NECTY features. Does NOT call Claude API in real time — too slow for a help search.

Onboarding Progress

Reads from Supabase workspaces to determine what the user has completed. Completion items: Connect 2+ Sources (check sources table count > 1), Write First Comment (check message_tracking for action=copied + type=comment), Send First DM (check message_tracking for action=copied + type=dm), Explore Opportunities (check if user has visited /opportunities — store in Supabase user_progress table), Schedule Onboarding Call (external link click, tracked via UTM).

SECTION 6 REUSABLE COMPONENT LIBRARY

Shared Components

Component	Props	Usage
KPICard	title, value, subtitle, trend (% + direction), icon, linkTo	All pages — dashboard, comments, DMs, insights, sources
FilterBar	filters: FilterConfig[], activeFilters, onChange	Opportunities, Comments, DMs, Insights pages
OpportunityRow / Card	opportunity: Opportunity, variant (row|card), onSelect, onCopy	Opportunities, Dashboard feed
MessageAngleCard	message: Message, onCopy, onRegenerate, showLikelihood	Comments page, DMs page, Opportunities panel
LiveFeedIndicator	lastUpdated, autoRefresh, onToggleRefresh	Dashboard, Opportunities, Comments, DMs
EmptyState	title, description, action: {label, onClick}	All pages when no data
SkeletonRow	columns: number	All list pages loading state
IntentBadge	level: high|medium|low	All opportunity/message displays
SignalTypeBadge	type: direct_need|recommendation|competitor|price|general	All opportunity displays
UrgencyBadge	urgency: immediate|this_week|general|none	Opportunities, Comments, DMs
AIScoreBadge	score: number	Opportunity rows
CopyButton	text: string, onCopy: () => void, variant: icon|button	Message angle cards
PlatformIcon	platform: string, size: sm|md|lg	All source/opportunity displays
ReplyLikelihoodBar	likelihood: number	Comment and DM angle cards
ToastNotification	message, type: success|error|info, duration	Copy feedback, save confirmations
SlidingPanel	isOpen, onClose, width, children	Selected opportunity panel on Opportunities page
TagInput	value: string[], onChange, placeholder	Settings, Sources keyword inputs
SearchBar	placeholder, value, onChange, shortcut	Opportunities (Cmd+K), Comments, DMs
WorkspaceSwitcher	workspace, onSwitch (future multi-workspace)	Top bar global component

Component	Props	Usage
NotificationDropdown	notifications: Notification[], onMarkRead	Top bar bell icon

KPICard Component

```
// components/shared/KPICard.tsx
interface KPICardProps {
  title: string;
  value: string | number;
  subtitle?: string;
  trend?: { value: number; direction: 'up' | 'down'; label?: string };
  icon?: React.ReactNode;
  linkTo?: string;
  className?: string;
}

export function KPICard({ title, value, subtitle, trend, icon, linkTo }: KPICardProps) {
  return (
    <div className="bg-white rounded-xl border border-gray-100 p-4 flex flex-col gap-1">
      <div className="flex items-center justify-between">
        <span className="text-xs font-medium text-gray-500 uppercase
tracking-wide">{title}</span>
        {icon && <span className="text-gray-400">{icon}</span>}
      </div>
      <div className="text-2xl font-bold text-gray-900">{value}</div>
      {subtitle && <div className="text-xs text-gray-500">{subtitle}</div>}
      {trend && (
        <div className={`text-xs font-medium flex items-center gap-1
          ${trend.direction === 'up' ? 'text-green-600' : 'text-red-500'}`}>
          {trend.direction === 'up' ? '↑' : '↓'} {trend.value}% {trend.label}
        </div>
      )}
      {linkTo && (
        <a href={linkTo} className="text-xs text-orange-500 hover:underline mt-1">
          View all →
        </a>
      )}
    </div>
  );
}
```

Data Fetching Pattern — Custom Hooks

```
// hooks/useOpportunities.ts
import { useQuery } from '@tanstack/react-query';
import { useFiltersStore } from '@/store/filters';

export function useOpportunities() {
  const filters = useFiltersStore(s => s.opportunityFilters);
```

```
  return useQuery({
    queryKey: ['opportunities', filters],
    queryFn: async () => {
      const params = new URLSearchParams(filters as Record<string, string>);
      const res = await fetch(`/api/airtable/opportunities?${params}`);
      if (!res.ok) throw new Error('Failed to fetch opportunities');
      return res.json() as Promise<Opportunity[]>;
    },
    staleTime: 60_000,       // 1 minute — opportunities can change frequently
    refetchInterval: 90_000, // Auto-refetch every 90s when autoRefresh is enabled
  });
}

// hooks/useTracking.ts
import { useMutation, useQueryClient } from '@tanstack/react-query';

export function useTrackMessage() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async (payload: TrackingPayload) => {
      await fetch('/api/tracking', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload)
      });
    },
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: ['performance'] });
    }
  });
}
```

SECTION 7 STRIPE INTEGRATION

Plans

Plan	Stripe Price ID	Features	Gating Logic
Trial	N/A (no payment)	7-day full access, limited opportunities (20/day)	workspaces.plan = trial AND created_at < 7 days ago
Pro	STRIPE_PRO_PRICE_ID	Unlimited opportunities, all pages, all features	workspaces.plan = pro AND plan_status = active
Agency	STRIPE_AGENCY_PRICE_ID	Multiple workspaces, team seats, priority support	workspaces.plan = agency

Stripe Webhook Handler

```
// app/api/webhooks/stripe/route.ts
export async function POST(req: NextRequest) {
  const body = await req.text();
  const signature = req.headers.get('stripe-signature')!;

  let event: Stripe.Event;
  try {
    event = stripe.webhooks.constructEvent(body, signature,
process.env.STRIPE_WEBHOOK_SECRET!);
  } catch (err) {
    return NextResponse.json({ error: 'Invalid signature' }, { status: 400 });
  }

  const supabase = createServiceClient();

  switch (event.type) {
    case 'checkout.session.completed': {
      const session = event.data.object as Stripe.CheckoutSession;
      await supabase.from('workspaces').update({
        plan: 'pro',
        plan_status: 'active',
        stripe_customer_id: session.customer as string
      }).eq('owner_id', session.metadata!.user_id);
      break;
    }
    case 'customer.subscription.deleted': {
      const sub = event.data.object as Stripe.Subscription;
      await supabase.from('workspaces').update({
        plan: 'trial', plan_status: 'cancelled'
      }).eq('stripe_customer_id', sub.customer as string);
      break;
    }
    case 'invoice.payment_failed': {
      const invoice = event.data.object as Stripe.Invoice;
```

```
      await supabase.from('workspaces').update({
        plan_status: 'past_due'
      }).eq('stripe_customer_id', invoice.customer as string);
      break;
    }
  }

  return NextResponse.json({ received: true });
}
```

Plan Gating Middleware

Add plan checking to the dashboard layout. If plan_status = past_due or cancelled, show an upgrade banner. If trial has expired (created_at > 7 days ago and plan = trial), gate all pages except /settings and /help with an upgrade modal.

```
// In /(dashboard)/layout.tsx Server Component:
const { data: workspace } = await supabase
  .from('workspaces')
  .select('plan, plan_status, created_at')
  .eq('owner_id', session.user.id)
  .single();

const trialExpired = workspace.plan === 'trial' &&
  new Date(workspace.created_at) < new Date(Date.now() - 7 * 24 * 60 * 60 * 1000);

if (trialExpired || workspace.plan_status === 'cancelled') {
  // Pass prop to layout to show upgrade gate
}
```

SECTION 8 SYSTEM INTEGRATION FLOWS

Opportunity Lifecycle

How a single opportunity flows from raw conversation to user action:

Stage	Where	What Happens
1. Scrape	Apify via Make.com	Public conversation captured from Reddit, Google, TikTok, etc.
2. Classify	Claude API Agent 1 via Make.com	Signal scored, intent classified, noise filtered, dm_viability set
3. Store	Airtable demand_signals	Raw signal written with all classification fields
4. Generate	Claude API Agent 2 via Make.com	3 comment + 3 DM variations generated as structured JSON
5. Store messages	Airtable messages	6 message records linked to opportunity
6. Cache to Supabase	Make.com → /api/webhooks/make	dashboard_cache updated, notification created if high urgency
7. Frontend display	Next.js reads Airtable	Opportunities page fetches via /api/airtable/opportunities
8. User action	Next.js → Supabase	Copy/Send action tracked via /api/tracking to message_tracking
9. Performance sync	Make.com Scenario 8	Weekly aggregation writes insights to performance_summary in Supabase

Copy/Paste Tracking Flow

This is the primary conversion signal NECTY collects in V1. Every copy action is tracked.

```
// COPY FLOW — triggered on Copy button click:

1. USER CLICKS COPY BUTTON
   → Component calls: navigator.clipboard.writeText(message.text)
   → Show "Copied!" toast (2 seconds)
   → Button enters checkmark state

2. TRACKING POST
   → POST /api/tracking
   → Body: { airtable_message_id, airtable_opp_id, message_type,
             angle_type, offer_used, platform, action: "copied" }
   → Server validates session, gets workspace_id
   → INSERT INTO message_tracking
   → UPDATE performance_summary (increment total_copied)

3. OPTIMISTIC UI UPDATE
   → Opportunity row shows "Copied" subtle indicator
   → Store copied message IDs in sessionStorage
   → Previously copied messages show different button state on page revisit

// MARK SENT FLOW — user manually marks after posting:
1. User clicks "Mark Sent" on a message (shown after copy)
2. POST /api/tracking with action: "sent"
3. If user later gets a reply, they click "Mark Replied"
4. POST /api/tracking with action: "replied"
5. performance_summary reply_rate recalculated server-side
```

Auto-Refresh Flow

```
// When auto-refresh is ON on Opportunities/Comments/DMs pages:

1. TanStack Query refetchInterval = 90_000 (90 seconds)
2. On refetch, fetch /api/airtable/opportunities with same filters
3. Compare returned IDs to currently displayed IDs
4. If new opportunities exist: show toast "3 new opportunities found"
5. Prepend new opportunities to top of list with a "New" highlight pulse
6. LiveFeedIndicator updates: "New conversations detected X seconds ago"

// Auto-refresh toggle state:
→ Stored in Zustand refreshStore
→ Persisted in localStorage (default: ON)
→ Toggle UI: green "Auto-refresh ON" / gray "Auto-refresh OFF"
```

Onboarding → Make.com → First Opportunity Flow

```
1. User completes /onboarding (all 6 steps)
2. Frontend POSTs to /api/onboarding/complete
3. /api/onboarding/complete calls Make.com Scenario 10 webhook with workspace payload
4. Make.com Scenario 10:
   a. Creates Airtable control_panel record with airtable_client_id
```

```
   b. Updates Supabase workspaces.airtable_client_id via service role
   c. Triggers keyword generation (Scenario 2)
   d. Triggers first scrape (Scenario 1)
5. After first scrape completes, Make.com fires pipeline_complete webhook
6. Frontend /api/webhooks/make updates dashboard_cache
7. User sees first opportunities on Dashboard/Opportunities page
   Time from onboarding complete to first opportunities: ~10–20 minutes

// During this wait period, Dashboard shows:
"NECTY is scanning your market for the first time.
 Your first opportunities will appear in about 15 minutes."
+ animated scanning indicator
+ link to /help for what to do while waiting
```

SECTION 9 PERFORMANCE, SECURITY & SCALABILITY

Performance Requirements

Metric	Target	How to Achieve
Dashboard initial load	< 1.5s	Read from dashboard_cache (Supabase) — never call Airtable on dashboard load
Opportunities page load	< 2s	Server-side Airtable fetch in API route, paginate at 50 records
Copy action	< 100ms feel	Optimistic UI — show Copied state before tracking API call completes
Filter change	< 800ms	TanStack Query caches previous filter results, new fetch debounced 300ms
Settings save	< 500ms	Supabase write is fast, show immediate success state
Insights page	< 2.5s	Parallel fetch from Supabase performance_summary + Airtable insights

Airtable Pagination

Airtable has a 100-record-per-request limit. For workspaces with many opportunities, implement cursor-based pagination:

```
export async function getOpportunities(clientId: string, offset?: string) {
  const result = await base('opportunities')
    .select({
      filterByFormula: `{client_id} = "${clientId}"`,
      sort: [{ field: 'opportunity_score', direction: 'desc' }],
      pageSize: 50,
      offset: offset  // Airtable cursor for next page
    })
    .firstPage();

  return {
    records: result.map(mapOpportunityRecord),
    nextOffset: result._offset  // pass to next call for pagination
  };
}

// Frontend: load more button or infinite scroll
// "Load More" is preferred over infinite scroll for V1 — simpler, less error-prone
```

Mobile Responsiveness Requirements

Page	Mobile Behavior
Dashboard	Single column. KPI cards in 2-col grid. Feed becomes card stack. No right panel.
Opportunities	Cards instead of rows. Selected opportunity = full-screen overlay with back button.
Comments	Cards stacked. Angle cards in vertical stack (not horizontal). Engagement strategy = collapsible bottom sheet.
DMs	Same as Comments. Message This Lead button = sticky bottom CTA.
Insights	Tables become horizontal scroll. Charts become single-column. AI Recommendations = collapsible section.
Sources	Source cards in single column. Search config = accordion sections.
Settings	Each section becomes an accordion. Save button sticky at bottom of each section.
Help	Guide cards in single column. Right panels (onboarding, AI recs) move below main content.

Security Best Practices

- NEXT_PUBLIC_ prefix only on env vars that are safe for the browser. Never expose AIRTABLE_API_KEY, SUPABASE_SERVICE_ROLE_KEY, STRIPE_SECRET_KEY, or MAKE_WEBHOOK_SECRET with NEXT_PUBLIC_.
- All Airtable reads go through server-side API routes only — the Airtable API key never leaves the server.
- Stripe webhook signature verification on every request to /api/webhooks/stripe.
- Make.com webhook HMAC signature verification on every request to /api/webhooks/make.
- RLS enabled on all Supabase tables — verified by testing as two different users.
- Content Security Policy headers set in next.config.js.
- Rate limiting on /api/tracking using Vercel Edge middleware (max 100 requests/minute per user).

V1 → V2 Migration Path

NECTY V1 uses Airtable as the intelligence pipeline database. V2 migrates this to Supabase for full data ownership, better querying, lower cost at scale, and real-time subscriptions. Plan this migration from day one.

V1 (Airtable)	V2 (Supabase)	Migration Complexity
opportunities table	opportunities table in Supabase	Low — schema already designed for Supabase
demand_signals table	demand_signals table in Supabase	Low
messages table	messages table in Supabase	Low
Make.com writes to Airtable	Make.com writes directly to Supabase via HTTP	Medium — update all Make.com HTTP modules
Frontend reads Airtable via API route	Frontend reads Supabase directly via SDK	Low — swap fetch calls for Supabase queries

V1 (Airtable)	V2 (Supabase)	Migration Complexity
dashboard_cache in Supabase	Stays in Supabase	None
Airtable views for filtering	Supabase RLS + PostgreSQL queries	Medium — rewrite filter logic
No real-time updates	Supabase Realtime for live feed	Low — add subscription on migration

Migration Preparation Steps (Build Now)

Use the same field names in Airtable as in the planned Supabase schema — this is already done in the automation brief.

Keep all Airtable reads behind /api/airtable/* routes — on migration, only these files change.

The message_tracking and performance_summary tables already live in Supabase — no migration needed.

When ready for V2: 1) Create Supabase tables with same schema, 2) Run Make.com migration to copy Airtable data, 3) Swap API routes to use Supabase client, 4) Disable Airtable reads.

SECTION 10 VERCEL DEPLOYMENT & LAUNCH CHECKLIST

Vercel Project Configuration

```
// next.config.js
const nextConfig = {
  images: {
    domains: ['avatars.githubusercontent.com', 'storage.googleapis.com'],
  },
  async headers() {
    return [
      {
        source: '/(.*)',
        headers: [
          { key: 'X-Frame-Options', value: 'DENY' },
          { key: 'X-Content-Type-Options', value: 'nosniff' },
          { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
        ]
      }
    ];
  }
};

// vercel.json
{
  "buildCommand": "next build",
  "outputDirectory": ".next",
  "framework": "nextjs",
  "regions": ["iad1"],
  "functions": {
    "app/api/webhooks/**": { "maxDuration": 30 }
  }
}
```

Pre-Launch Checklist

Supabase

- All 8 tables created with correct schema
- RLS enabled and tested on all tables — verified by logging in as two different users
- Supabase Auth email templates customized with NECTY branding
- Supabase Auth redirect URLs configured for Vercel domain
- Service role key stored in Vercel env vars only — not in code

Stripe

- Products and prices created in Stripe dashboard
- Webhook endpoint configured: https://app.necty.io/api/webhooks/stripe
- Webhook events enabled: checkout.session.completed, customer.subscription.deleted, invoice.payment_failed

- Webhook secret stored in STRIPE_WEBHOOK_SECRET env var
- Test mode webhooks tested end-to-end before switching to live mode

Airtable Integration

- Airtable API key stored in AIRTABLE_API_KEY env var — server only, no NEXT_PUBLIC_ prefix
- All /api/airtable/* routes tested against real Airtable data
- Airtable field names verified to match exactly the names expected in mapOpportunityRecord()
- airtable_client_id correctly written to Supabase workspaces on onboarding completion

Make.com Webhooks

- Make.com pipeline_complete event tested end-to-end — dashboard_cache updates correctly
- Make.com HMAC secret matches MAKE_WEBHOOK_SECRET env var
- Onboarding webhook URL (MAKE_ONBOARDING_WEBHOOK_URL) tested — triggers Scenario 10

Frontend

- All 8 pages render correctly on desktop (1280px), tablet (768px), mobile (390px)
- Loading states appear on all pages
- Empty states appear when no data — never a blank white page
- Error states handled — no unhandled Promise rejections in production
- Copy to clipboard tested on Safari (iOS) — requires specific permission handling
- All form inputs validated with Zod — no invalid data reaches Supabase
- Onboarding flow tested end-to-end: signup → onboarding → first opportunity visible

Performance

- Lighthouse score > 80 on all pages (Performance, Accessibility, Best Practices)
- No Airtable calls happening on Dashboard page load — reads from dashboard_cache
- TanStack Query staleTime and refetchInterval set correctly per page
- Images optimized — all logos and platform icons use next/image

NECTY V1 — Full-Stack Developer Build Document Complete

Build exactly what is in this document. Nothing more. Nothing less. Ship fast.

SECTION 11 AIRTABLE DATA REFERENCE FOR THE FRONTEND

This section is the complete Airtable schema reference formatted specifically for the frontend developer. It shows what each table contains, what data types to expect, what TypeScript types to define, and how to query each table from the /api/airtable/* routes. The automation contractor builds and maintains the actual Airtable base — you read from it.

> **How to Read This Reference**
>
> This document shows exactly which Airtable tables the frontend reads from, what each field contains, and what TypeScript type to use when mapping it. The frontend never writes to Airtable — all writes go to Supabase. Your Airtable reads go through /api/airtable/* server-side API routes only.
>
> Orange field names are primary or foreign keys. All fields use snake_case. Linked record fields return the linked record ID as a string — resolve them server-side before returning to the client.

Table Relationship Map

All tables link back to control_panel via client_id. This is how workspace isolation works — every Airtable query must include a filter on client_id matching the workspace's airtable_client_id stored in Supabase.

```
control_panel (one workspace)
  ├── sources            (many → what platforms are being scraped)
  ├── demand_signals     (many → raw scraped conversations)
  ├── opportunities      (many → cleaned, scored, ready for action)
  ├── messages           (many → generated comments + DMs per opportunity)
  ├── message_performance (many → outcome tracking per message)
  ├── trends             (many → aggregated market trend summaries)
  ├── competitors        (many → competitor mentions detected)
  └── insights           (many → AI-generated performance recommendations)

opportunities ←── demand_signals  (each opportunity links to its source signal)
messages      ←── opportunities   (each message links to its parent opportunity)
messages      ←── demand_signals  (each message also links to source signal)
message_performance ←── messages, opportunities, control_panel
```

> **Mandatory Filter on Every Airtable Query**
>
> Every query to every table MUST include: filterByFormula: `{client_id} = "${airtableClientId}"`
>
> The airtableClientId comes from Supabase: workspaces.airtable_client_id for the authenticated user's workspace.
>
> Omitting this filter would allow one user's queries to return another user's data.
>
> Format: WS-001, WS-002, WS-003 etc. This is set by Make.com on onboarding.

control_panel

Workspace configuration. One record per workspace. Read this first on app init to get workspace context.

The frontend reads this table once on session start to get the workspace's configuration context (industry, city, booking_link, etc.). Cache the result in Zustand workspaceStore. Do not re-fetch on every page.

Field Name	Airtable Type	What the Frontend Uses This For
client_id	single line text	Primary key. Foreign key for all other table queries. Store in workspaceStore as airtableClientId.
business_name	single line text	Display in sidebar, Settings page header, and message generation context.
industry	single select	Display in workspace switcher (top bar). Used as filter context label.
custom_industry	single line text	If set, display instead of industry. Falls back to industry if null.
city	single line text	Display in top bar city/industry switcher.
state	single line text	Pair with city for location display.
service_area_radius	number	Display in Settings page service area field. Integer (miles).
booking_link	url	Used in message previews where CTA links appear. Show as clickable in Settings.
default_offer	single select	Display in Settings → AI Preferences. Shown on message cards as Offer Used tag.
preferred_tone	single select	Display in Settings → AI Preferences. Shown as active tone selection.
platforms	multi-select	Array of connected platform names. Used on Sources page to show connected status.
is_active	checkbox (boolean)	If false, show workspace as paused. Gray out live feed indicators.
last_run_at	date/time	Display on Sources page as last pipeline run time. Show relative time (e.g. "8 min ago").

sources

Specific platforms and search configurations being monitored. Powers the Sources page.

Read all sources for the workspace to populate the Sources page platform cards. Each record represents one platform/search configuration.

Field Name	Airtable Type	What the Frontend Uses This For
source_id	single line text	Primary key. Use as React key prop on source card list.
client_id	linked record	Filter key. Always filter by workspace client_id.
source_name	single line text	Display as label on source card (e.g. "r/Dallas_roofing").
platform	single select	Maps to platform icon. Values: Reddit, TikTok, Instagram, Facebook, Google Maps, YouTube, X, Nextdoor.
source_type	single select	Display as source type badge: Subreddit, Hashtag, Location search, Facebook group, Keyword search.

Field Name	Airtable Type	What the Frontend Uses This For
status	single select	Active → green badge, Paused → gray badge, Error → red badge. Show on source card.
last_scraped_at	date/time	Display as "Last sync X min ago" on source card. Use date-fns formatDistanceToNow().
keyword_query	single line text	Display on source Configure modal as the active search keyword.
city	single line text	Show on source card as location context.
notes	long text	Display in source Configure modal as internal notes. Not shown on main card.

opportunities

Cleaned, scored, actionable opportunities. Primary data source for Opportunities, Comments, and DMs pages.

This is the highest-traffic Airtable read. The Opportunities, Comments, and DMs pages all read from this table with different filters. Paginate at 50 records. Sort by opportunity_score DESC by default.

Field Name	Airtable Type	What the Frontend Uses This For
opportunity_id	single line text	Primary key. Pass as airtable_opp_id in all tracking POST calls.
client_id	linked record	Mandatory filter field.
signal_id	linked record	Links to source demand signal. Not displayed in UI but needed for message lookups.
title	single line text	Main display text on opportunity card and row. e.g. "Roof leaking after storm in Dallas".
platform	single select	Maps to PlatformIcon component. Use for icon and badge display.
city	single line text	Display on opportunity card as location. Pair with state.
industry	single line text	Display as category label on card.
source_link	url	Used in Open Thread / Open Source button. Open in new tab.
original_context	long text	Full post text. Display in Selected Opportunity Panel as conversation preview.
pain_point	long text	Short AI summary. Display as subtext on opportunity card and in panel.
signal_type	single select	Maps to SignalTypeBadge. Values: direct_need, problem_statement, competitor_dissatisfaction, recommendation_request, contextual_hook.
intent_level	single select	Maps to IntentBadge color. High=orange, Medium=yellow, Low=gray.
urgency	single select	Maps to UrgencyBadge. Values: immediate, this_week, general, none.
sentiment	single select	Display as subtle tag. Values: positive, neutral, negative.

Field Name	Airtable Type	What the Frontend Uses This For
competitor_mentioned	single line text	If not null, show Competitor Mention badge on card. Display name in panel.
opportunity_score	number	Display as AIScoreBadge (circle with number). 0–100. Color: 80+=green, 60–79=yellow, <60=gray.
recommended_action	single select	Display as Suggested Approach on card. Values: comment_first, dm_first, both, monitor.
status	single select	Use to show responded state on card. Values: new, comment_generated, dm_generated, message_used, replied, clicked, booked, dismissed.
created_at	date/time	Display as "Posted X min ago" using formatDistanceToNow(). Sort by this for recent feed.

Filter logic for each page:

Page	Airtable Filter to Add
Opportunities	No extra filter — show all non-dismissed opportunities
Comments	AND({comment_viability} = 1) — only comment-viable signals
DMs	AND({dm_viability} = 1) — only DM-viable signals
Dashboard feed	Top 5 by opportunity_score, created_at within last 24h

messages

All AI-generated comments and DMs. 6 records per opportunity (3 comments + 3 DMs). Powers the angle cards on Comments, DMs, and Opportunities pages.

When a user opens an opportunity (on any page), fetch all messages linked to that opportunity_id. Returns up to 6 records. Render as CommentAngleCard or DMAngleCard components based on message_type.

Field Name	Airtable Type	What the Frontend Uses This For
message_id	single line text	Primary key. Pass as airtable_message_id in all tracking POST calls. Critical for performance attribution.
opportunity_id	linked record	Filter by this to fetch all messages for a given opportunity.
client_id	linked record	Mandatory filter field.
message_type	single select	comment or dm. Use to render CommentAngleCard vs DMAngleCard component.
variation_number	number	1, 2, or 3. Use to order cards. Display as card number badge.
angle_type	single select	Comments: balanced, friendly, direct. DMs: soft_assist, consultative, direct_offer. Map to angle badge label and color.
message_text	long text	The full generated message. This is what gets copied to clipboard on Copy button click.

Field Name	Airtable Type	What the Frontend Uses This For
offer_used	single select	Display as Offer tag on message card. e.g. "Free inspection", "24-hour quote".
cta_type	single select	Display as CTA tag on message card. e.g. "booking_link", "call_now".
booking_link_used	url	The specific UTM-tagged booking link inserted into this message. Do not display in UI but include in copied message_text.
prompt_version	single line text	Display as small metadata tag on message card. e.g. "v1.0". Helps identify which prompt generation this came from.
personalization_fields_used	multi-select	Display as small tags: location, urgency, competitor_name, etc. Shows user what was personalized.
status	single select	draft, copied, sent, posted, replied, clicked, booked, archived. Update in Supabase message_tracking — do not write back to Airtable.
generated_at	date/time	Display as "Generated X min ago" in message card metadata.

Angle type to display label mapping:

angle_type value	Display Label	Badge Color
balanced	Balanced	Blue
friendly	Friendly	Green
direct	Direct Offer	Orange
soft_assist	Soft Assist	Purple
consultative	Consultative	Teal
direct_offer	Direct Offer	Orange

message_performance

Outcome tracking per message. Automation contractor uses this. Frontend reads conversion_score for Insights page aggregations only.

The frontend does NOT write to this table. Performance tracking is handled by Make.com (Scenario 7). The frontend writes user actions to Supabase message_tracking instead, and Make.com syncs aggregated data back to Supabase performance_summary weekly.

Field Name	Airtable Type	What the Frontend Uses This For
performance_id	single line text	Primary key. Not used directly by frontend.
message_id	linked record	Links to messages table. Not used directly by frontend.
conversion_score	number	Only field the frontend reads. Used in Insights page for aggregate performance data if reading directly from Airtable. Prefer Supabase performance_summary instead.

Field Name	Airtable Type	What the Frontend Uses This For
angle_type	single select	Read for Insights aggregation if needed. Prefer Supabase performance_summary.best_angle.
offer_used	single select	Read for Insights aggregation if needed. Prefer Supabase performance_summary.best_offer.
platform	single select	Read for platform performance breakdown if needed.

Frontend Preference for Performance Data

For the Insights page, prefer reading from Supabase performance_summary (synced weekly by Make.com) over querying Airtable message_performance directly.
Supabase is faster, already aggregated, and does not count toward Airtable API rate limits.
Only query Airtable message_performance directly if you need granular per-message data not available in the Supabase summary.

trends

Market-level aggregated demand data. Powers trend themes on the Insights and Dashboard pages.

Field Name	Airtable Type	What the Frontend Uses This For
trend_id	single line text	Primary key.
client_id	linked record	Mandatory filter field.
theme	single line text	Display as trending topic label. e.g. "Storm damage roofing", "Emergency HVAC".
mention_count	number	Display as count badge next to theme.
high_intent_count	number	Display as high-intent subset count.
high_intent_percentage	number (percent)	Display as percentage badge.
trend_direction	single select	Rising → green up arrow, Stable → gray dash, Declining → red down arrow.
best_angle	single select	Display as recommended angle for this trend theme.
best_offer	single select	Display as recommended offer for this trend theme.
period_start	date	Display as trend period range.
period_end	date	Display as trend period range.
updated_at	date/time	Display as "Updated X ago" on trend card.

competitors

Competitor mentions detected in demand signals. Powers the competitor dissatisfaction filter and opportunity tagging.

Field Name	Airtable Type	What the Frontend Uses This For
competitor_id	single line text	Primary key.
client_id	linked record	Mandatory filter field.
competitor_name	single line text	Display in competitor mention badge on opportunity card.
platform	single select	Display as source platform on competitor mention.
source_link	url	Used in Open Thread button on competitor opportunity.
mention_text	long text	Display as original post context in competitor opportunity panel.
sentiment	single select	Negative → show Dissatisfied badge (high priority). Neutral or positive → lower priority badge.
complaint_detected	checkbox (boolean)	If TRUE, show Act Now or High Priority flag on opportunity. This is a high-conversion signal.
opportunity_gap	long text	Display as AI insight text in the competitor opportunity panel. e.g. "Competitor failed to show up — positioning opportunity."
created_at	date/time	Sort by to surface most recent competitor mentions first.

insights

AI-generated optimization recommendations. Powers the AI Recommendations panel on the Insights page.

Field Name	Airtable Type	What the Frontend Uses This For
insight_id	single line text	Primary key. Use as React key on insight card list.
client_id	linked record	Mandatory filter field.
insight_type	single select	Maps to insight card icon and color. Values: angle_winner, offer_winner, platform_pattern, prompt_improvement, low_signal_warning.
summary	long text	Main text of the AI insight card. Plain English. Display as body text.
recommended_action	long text	Display as call-to-action text under summary. e.g. "Use direct_offer DM angle first on high-intent storm damage posts."
supporting_metric	single line text	Display as metric badge. e.g. "CTR: 35% vs 22%".
platform	single select	If set, show platform icon on insight card to indicate platform-specific finding.
angle_type	single select	If set, show angle type badge on insight card.
offer_used	single select	If set, show offer badge on insight card.
confidence_score	number	Display as confidence bar (0–100). Only show insights with confidence_score >= 60.
created_at	date/time	Sort by to show most recent insights first. Display as "Generated X days ago".

TypeScript Type Definitions

Use these types when mapping Airtable API responses to frontend data. Define them in types/opportunity.ts, types/message.ts, and types/insights.ts.

```
// types/opportunity.ts
export type IntentLevel = "High" | "Medium" | "Low";
export type Urgency = "immediate" | "this_week" | "general" | "none";
export type SignalType =
  | "direct_need" | "problem_statement" | "competitor_dissatisfaction"
  | "recommendation_request" | "contextual_hook";
export type OpportunityStatus =
  | "new" | "comment_generated" | "dm_generated" | "message_used"
  | "replied" | "clicked" | "booked" | "dismissed";

export interface Opportunity {
  opportunity_id: string;
  title: string;
  platform: string;
  city: string;
  industry: string;
  source_link: string;
  original_context: string;
  pain_point: string;
  signal_type: SignalType;
  intent_level: IntentLevel;
  urgency: Urgency;
  sentiment: "positive" | "neutral" | "negative";
  competitor_mentioned: string | null;
  opportunity_score: number;          // 0-100
  recommended_action: "comment_first" | "dm_first" | "both" | "monitor";
  status: OpportunityStatus;
  created_at: string;                 // ISO date string
}

// types/message.ts
export type MessageType = "comment" | "dm";
export type AngleType =
  | "balanced" | "friendly" | "direct"            // comment angles
  | "soft_assist" | "consultative" | "direct_offer"; // DM angles

export interface Message {
  message_id: string;
  opportunity_id: string;
  message_type: MessageType;
  variation_number: 1 | 2 | 3;
  angle_type: AngleType;
  message_text: string;
  offer_used: string;
  cta_type: string;
  booking_link_used: string;
  prompt_version: string;
  personalization_fields_used: string[];
  generated_at: string;
}

// types/insights.ts
export type InsightType =
  | "angle_winner" | "offer_winner" | "platform_pattern"
  | "prompt_improvement" | "low_signal_warning";
```

```
export interface Insight {
  insight_id: string;
  insight_type: InsightType;
  summary: string;
  recommended_action: string;
  supporting_metric: string;
  platform: string | null;
  angle_type: AngleType | null;
  offer_used: string | null;
  confidence_score: number;   // only display if >= 60
  created_at: string;
}
```

Airtable Query Patterns by Page

These are the exact query structures to use in each /api/airtable/* route. Every query must include the client_id filter. Use the Airtable npm package server-side.

```
// /api/airtable/opportunities/route.ts
// Used by: Opportunities page, Dashboard feed
const filters = [`{client_id} = "${clientId}"`];
if (platform !== "all")    filters.push(`{platform} = "${platform}"`);
if (urgency !== "all")     filters.push(`{urgency} = "${urgency}"`);
if (intentLevel !== "all") filters.push(`{intent_level} = "${intentLevel}"`);
if (signalType !== "all")  filters.push(`{signal_type} = "${signalType}"`);
if (minScore)              filters.push(`{opportunity_score} >= ${minScore}`);
filters.push(`{status} != "dismissed"`);
const formula = `AND(${filters.join(", ")})`;

// /api/airtable/comments/route.ts
// Used by: Comments page
const formula = `AND({client_id} = "${clientId}", {comment_viability} = 1, {status} !=
"dismissed")`;

// /api/airtable/dms/route.ts
// Used by: DMs page
const formula = `AND({client_id} = "${clientId}", {dm_viability} = 1, {status} !=
"dismissed")`;

// /api/airtable/opportunities/[id]/messages/route.ts
// Used by: Selected Opportunity Panel, Comments angle cards, DMs angle cards
const formula = `AND({client_id} = "${clientId}", {opportunity_id} = "${oppId}")`;
// Returns up to 6 records. Sort by variation_number ASC.

// /api/airtable/insights/route.ts
// Used by: Insights page AI Recommendations panel
const formula = `AND({client_id} = "${clientId}", {confidence_score} >= 60)`;
// Sort by created_at DESC. Limit 10.
```

> **Airtable API Rate Limits — Know These Before You Build**
>
> Airtable free plan: 5 requests per second. Pro plan: 5 requests per second per base.

Max records per request: 100. Use pageSize: 50 for opportunity lists to stay under limits.
TanStack Query staleTime of 60 seconds prevents re-fetching on every navigation.
The Dashboard page must NEVER call Airtable — it reads from Supabase dashboard_cache only.
If a user applies rapid filter changes, debounce the Airtable API call by 400ms to avoid rate limit hits.
Implement exponential backoff retry (max 3 attempts) on any Airtable 429 response.

NECTY V1 — Full-Stack Developer Build Document Complete

Build exactly what is in this document. Ship fast. Nothing more, nothing less.